

GUIA PRÁTICO

Microsoft Fabric

Implementação Bronze → Silver → Gold

Projeto: Pedidos, Consumos e Faturação

Lakehouse: LH_Pedidos_Consumos_Faturacao

Camada	Finalidade
Bronze	Preservação dos dados brutos e dos incrementais
Silver	Limpeza, tipificação, deduplicação e qualidade
Gold	Modelo dimensional preparado para análise

Inclui scripts PySpark, resultados, erros reais, correções e evidências visuais da implementação.

Elaborado para estudo e demonstração prática da certificação Microsoft Fabric Data Engineer Associate (DP-700).

Índice

- 1. Objetivo e arquitetura
- 2. Preparação do Lakehouse e notebooks
- 3. Camada Bronze
- 4. Transformação Bronze → Silver
- 5. Controlo de qualidade na Silver
- 6. Modelação Silver → Gold
- 7. Qualidade e quarentena na Gold
- 8. Erros encontrados e resolução
- 9. Validação final
- 10. Próximos passos

1. Objetivo e arquitetura

O objetivo foi construir uma prova de conceito de engenharia de dados no Microsoft Fabric, utilizando uma arquitetura Medallion. Os dados de clientes, unidades de consumo, consumos, pedidos e faturas são preservados na Bronze, tratados na Silver e organizados num modelo dimensional na Gold.

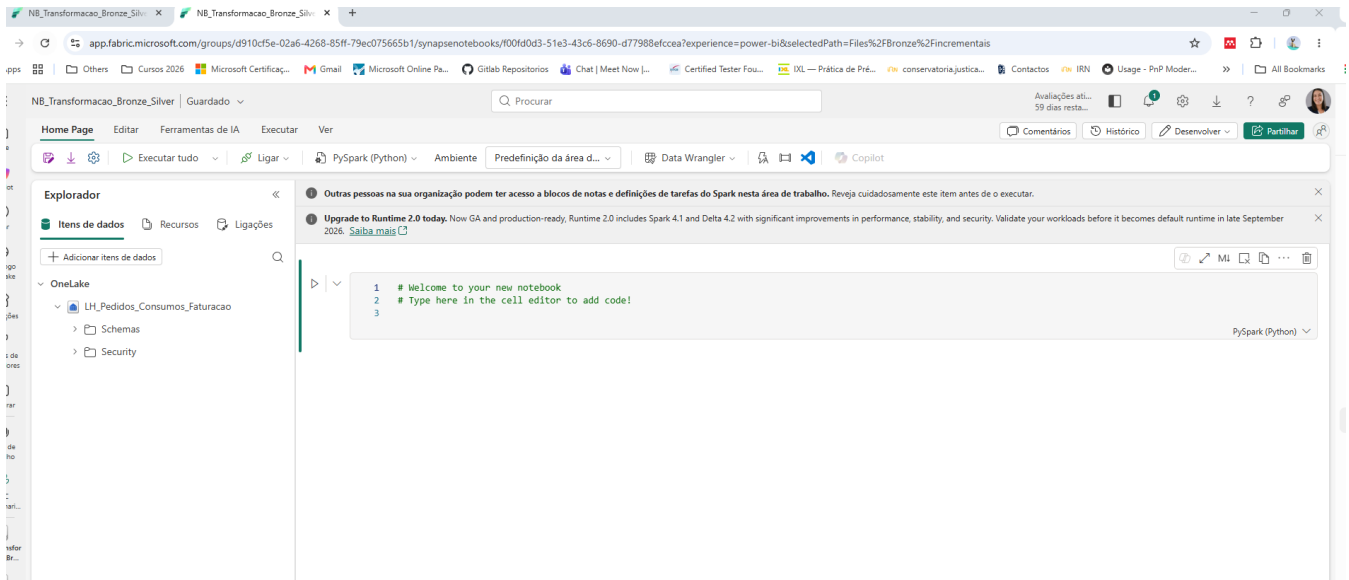
Fluxo	Descrição
Bronze	Ficheiros CSV e JSONL, incluindo cargas incrementais.
Silver	Tabelas Delta limpas, tipificadas, deduplicadas e validadas.
Gold	Dimensões Cliente, Unidade e Data; factos Consumo, Faturação e Pedido.
Consumo	Modelo semântico Direct Lake e relatórios Power BI.

Princípio central: os dados brutos não são alterados. Os registos inválidos são preservados em tabelas de rejeitados, garantindo rastreabilidade e auditoria.

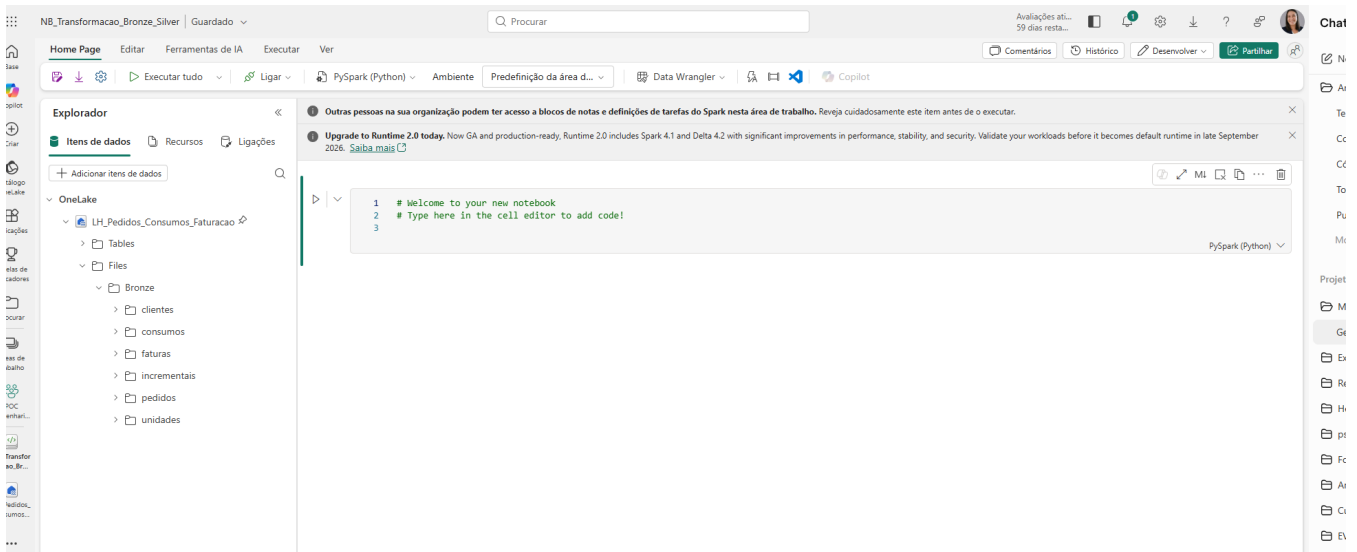
2. Preparação do Lakehouse e notebooks

2.1 Associar o Lakehouse correto

O notebook deve ser associado ao item Lakehouse e não ao SQL analytics endpoint. O indicador visual correto é a existência das áreas Tables e Files.



Tentativa inicial: item incorreto, apresentando apenas Schemas e Security.



Associação correta: o Lakehouse apresenta Tables, Files e a estrutura Bronze.

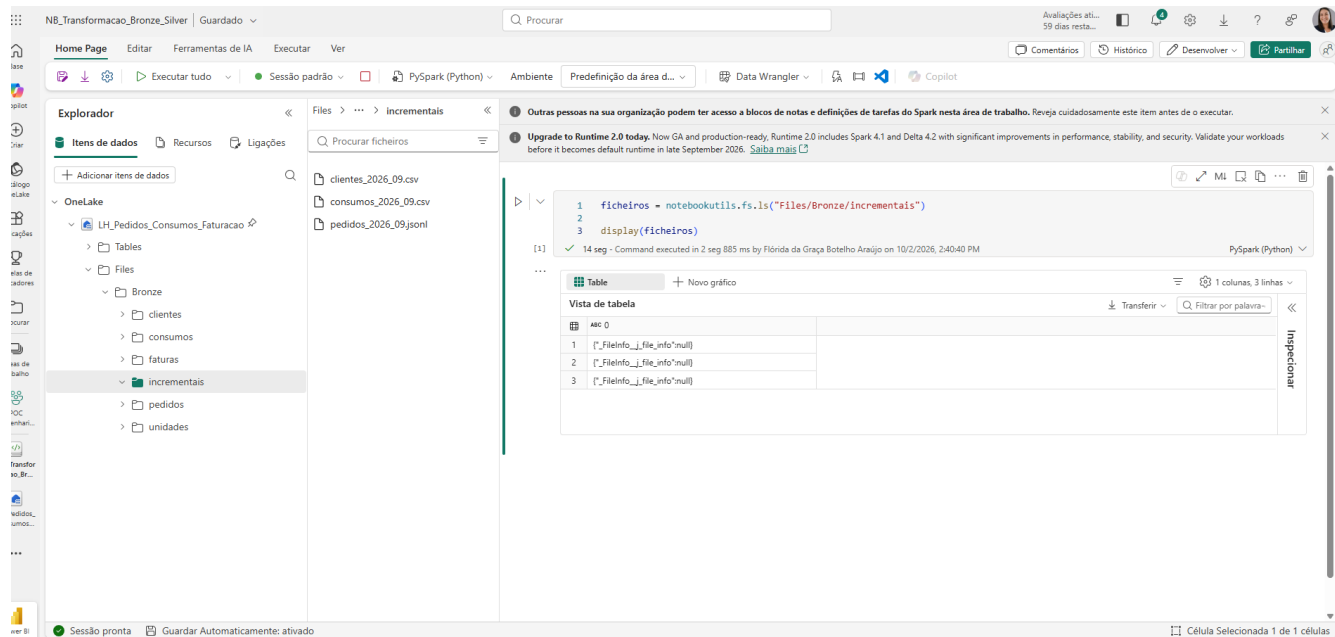
2.2 Organização utilizada

Lakehouse: LH_Pedidos_Consumos_Faturacao
Files/Bronze/clientes
Files/Bronze/consumos
Files/Bronze/faturas
Files/Bronze/incrementais
Files/Bronze/pedidos
Files/Bronze/unidades

Foram utilizados dois notebooks: NB_Transformacao_Bronze_Silver e NB_Modelacao_Silver_Gold.

3. Camada Bronze

A Bronze contém os ficheiros de origem sem transformação. A pasta incrementais contém três cargas adicionais.



The screenshot shows the Microsoft Fabric notebook environment. In the left-hand 'Explorador' (Explorer) pane, the 'Files' view is active, showing a directory structure under 'OneLake'. The 'incrementais' folder is selected, and its contents are listed: 'clientes_2026_09.csv', 'consumos_2026_09.csv', and 'pedidos_2026_09.json'. The main workspace displays a code cell with the following Python code:

```
1 ficheiros = notebookutils.fs.ls("Files/Bronze/incrementais")
2
3 display(ficheiros)
```

Below the code cell, a 'Table' view shows the output of the command. It has 1 column and 3 rows:

fileInfo_1_file_info
["_FileInfo_1_file_info"]
["_FileInfo_2_file_info"]
["_FileInfo_3_file_info"]

Conteúdo da pasta Bronze/incrementais: clientes, consumos e pedidos.

3.1 Listar ficheiros incrementais

```
ficheiros = notebookutils.fs.ls("Files/Bronze/incrementais")
display(ficheiros)
```

3.2 Ler os ficheiros

```
df_clientes_inc = (
    spark.read.option("header", "true")
              .option("inferSchema", "true")
              .csv("Files/Bronze/incrementais/clientes_2026_09.csv")
)

df_consumos_inc = (
    spark.read.option("header", "true")
              .option("inferSchema", "true")
              .csv("Files/Bronze/incrementais/consumos_2026_09.csv")
)

df_pedidos_inc = spark.read.json(
    "Files/Bronze/incrementais/pedidos_2026_09.jsonl"
)

display(df_clientes_inc)
display(df_consumos_inc)
display(df_pedidos_inc)
```

Outras pessoas na sua organização podem ter acesso a blocos de notas e definições de tarefas do Spark nesta área de trabalho. Reveja cuidadosamente este item antes de o executar.

Upgrade to Runtime 2.0 today. Now GA and production-ready, Runtime 2.0 includes Spark 4.1 and Delta 4.2 with significant improvements in performance, stability, and security. Validate your workloads before it becomes default runtime in late September 2026. [Saiba mais](#)

```

28 print("Pedidos:")
29 display(df_pedidos_inc)

```

17 seg - Command executed in 17 seg 442 ms by Flórida da Graça Botelho Araújo on 10/2/2026, 2:43:10 PM

Tarefas do Spark (8 de 8 com êxito) Resources Registo

Cientes:

ABC ClienteID	ABC NomeCliente	ABC Concelho	ABC Segmento	DataAdesao	ABC EstadoCliente	UltimaAtualizacao
1	CLI00501	Cliente Ficticio 00...	Vila Franca do...	Doméstico	2026-09-09	Ativo
2	CLI00502	Cliente Ficticio 00...	Povoação	Doméstico	2026-09-10	Ativo
3	CLI00503	Cliente Ficticio 00...	Nordeste	Doméstico	2026-09-04	Ativo
4	CLI00504	Cliente Ficticio 00...	Vila Franca do...	Doméstico	2026-09-13	Ativo
5	CLI00505	Cliente Ficticio 00...	Vila Franca do...	Serviços	2026-09-10	Ativo
6	CLI00506	Cliente Ficticio 00...	Vila Franca do...	Administração ...	2026-09-15	Ativo
7	CLI00507	Cliente Ficticio 00...	Lagoa	Administração ...	2026-09-03	Ativo
8	CLI00508	Cliente Ficticio 00...	Lagoa	Serviços	2026-09-06	Ativo
9	CLI00509	Cliente Ficticio 00...	Povoação	Serviços	2026-09-11	Ativo
10	CLI00510	Cliente Ficticio 00...	Lagoa	Serviços	2026-09-13	Ativo
11	CLI00010	Cliente Ficticio 00...	Lagoa	Serviços	2022-03-24	Ativo

Consumos:

ABC ConsumoID	ABC UnidadeID	DataLeitura	Periodo	12 QuantidadeKWh	ABC TipoLeitura	DataIngessao
---------------	---------------	-------------	---------	------------------	-----------------	--------------

Exemplo da leitura dos clientes e consumos incrementais.

3.3 Verificar schemas e totais

```

df_clientes_inc.printSchema()
print("Total:", df_clientes_inc.count())
df_consumos_inc.printSchema()
print("Total:", df_consumos_inc.count())
df_pedidos_inc.printSchema()
print("Total:", df_pedidos_inc.count())

```

Conjunto	Total incremental	Observação
Clientes	11	Datas já inferidas
Consumos	800	QuantidadeKWh como double
Pedidos	200	Datas inicialmente como string

4. Transformação Bronze → Silver

4.1 Ler os dados-base

```
df_clientes_base = spark.read.option("header", "true").option("inferSchema", "true").csv("Files/Bronze/clientes/clientes.csv")
df_consumos_base = spark.read.option("header", "true").option("inferSchema", "true").csv("Files/Bronze/consumos/consumos.csv")
df_pedidos_base = spark.read.json("Files/Bronze/pedidos/pedidos.jsonl")

df_clientes_total = df_clientes_base.unionByName(df_clientes_inc, allowMissingColumns=True)
df_consumos_total = df_consumos_base.unionByName(df_consumos_inc, allowMissingColumns=True)
df_pedidos_total = df_pedidos_base.unionByName(df_pedidos_inc, allowMissingColumns=True)
```

4.2 Limpeza e conversão de tipos

```
from pyspark.sql import functions as F
from pyspark.sql.window import Window

df_clientes_silver = (
    df_clientes_total
    .withColumn("ClienteID", F.trim("ClienteID"))
    .withColumn("NomeCliente", F.trim("NomeCliente"))
    .withColumn("DataAdesao", F.to_date("DataAdesao"))
    .withColumn("UltimaAtualizacao", F.to_timestamp("UltimaAtualizacao"))
    .filter(F.col("ClienteID").isNotNull())
)

df_consumos_silver = (
    df_consumos_total
    .withColumn("ConsumoID", F.trim("ConsumoID"))
    .withColumn("UnidadeID", F.trim("UnidadeID"))
    .withColumn("DataLeitura", F.to_date("DataLeitura"))
    .withColumn("QuantidadeKWh", F.col("QuantidadeKWh").cast("double"))
    .withColumn("DataIngestao", F.to_timestamp("DataIngestao"))
    .filter(F.col("ConsumoID").isNotNull())
)

df_pedidos_silver = (
    df_pedidos_total
    .withColumn("PedidoID", F.trim("PedidoID"))
    .withColumn("UnidadeID", F.trim("UnidadeID"))
    .withColumn("DataAbertura", F.to_timestamp("DataAbertura"))
    .withColumn("DataConclusao", F.to_timestamp("DataConclusao"))
    .withColumn("UltimaAtualizacao", F.to_timestamp("UltimaAtualizacao"))
    .filter(F.col("PedidoID").isNotNull())
)
```

4.3 Deduplicar conservando a versão mais recente

```
janela_clientes = Window.partitionBy("ClienteID").orderBy(F.col("UltimaAtualizacao").desc_nulls_last())
df_clientes_silver = (df_clientes_silver
    .withColumn("_rn", F.row_number().over(janela_clientes))
    .filter(F.col("_rn") == 1).drop("_rn"))

janela_consumos = Window.partitionBy("ConsumoID").orderBy(F.col("DataIngestao").desc_nulls_last())
df_consumos_silver = (df_consumos_silver
    .withColumn("_rn", F.row_number().over(janela_consumos))
    .filter(F.col("_rn") == 1).drop("_rn"))

janela_pedidos = Window.partitionBy("PedidoID").orderBy(F.col("UltimaAtualizacao").desc_nulls_last())
df_pedidos_silver = (df_pedidos_silver
    .withColumn("_rn", F.row_number().over(janela_pedidos))
    .filter(F.col("_rn") == 1).drop("_rn"))
```

4.4 Gravar tabelas Delta

```
df_clientes_silver.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable("silver_clientes")
df_consumos_silver.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable("silver_consumos")
df_pedidos_silver.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable("silver_pedidos")
```

... Clientes Silver: 510
Consumos Silver: 20000
Pedidos Silver: 10200

Table

+ Novo gráfico

7 colunas, 510 linhas

Vista de tabela

Transferir

Filtrar por palavra-

	ABC ClientID	ABC NomeCliente	ABC Concelho	ABC Segmento	DataAdesao	ABC EstadoCliente	UltimaAtualizacao	
1	CLI00001	Cliente Fictício 00...	Ponta Delgada	Doméstico	2019-04-03	Ativo	2025-03-26 16:08:01	
2	CLI00002	Cliente Fictício 00...	Ribeira Grande	Comércio	2016-08-15	Ativo	2026-01-20 02:57:24	
3	CLI00003	Cliente Fictício 00...	Ribeira Grande	Comércio	2025-01-19	Ativo	2026-03-25 00:31:34	
4	CLI00004	Cliente Fictício 00...	Ponta Delgada	Doméstico	2021-10-21	Ativo	2025-03-07 15:17:49	
5	CLI00005	Cliente Fictício 00...	Ponta Delgada	Administração ...	2016-06-19	Ativo	2026-06-15 21:25:24	
6	CLI00006	Cliente Fictício 00...	Ponta Delgada	Doméstico	2022-07-21	Ativo	2026-04-11 20:10:54	
7	CLI00007	Cliente Fictício 00...	Vila Franca do...	Serviços	2025-04-18	Ativo	2026-06-23 19:18:29	
8	CLI00008	Cliente Fictício 00...	Lagoa	Administração ...	2017-09-17	Ativo	2025-07-06 23:35:53	
9	CLI00009	Cliente Fictício 00...	Vila Franca do...	Doméstico	2020-10-03	Ativo	2026-01-12 20:31:56	
10	CLI00010	Cliente Fictício 00...	Lagoa	Serviços	2022-03-24	Ativo	2026-09-15 09:30:00	
11	CLI00011	Cliente Fictício 00...	Nordeste	Doméstico	2019-11-18	Ativo	2025-12-18 20:09:43	
12	CLI00012	Cliente Fictício 00...	Ponta Delgada	Comércio	2023-06-01	Ativo	2026-04-24 09:09:52	
13	CLI00013	Cliente Fictício 00...	Nordeste	Comércio	2018-02-13	Ativo	2025-02-16 23:00:56	
14	CLI00014	Cliente Fictício 00...	Ponta Delgada	Doméstico	2024-11-24	Ativo	2026-06-10 13:05:03	
15	CLI00015	Cliente Fictício 00...	Ponta Delgada	Doméstico	2022-04-28	Ativo	2025-04-16 08:33:48	

Célula Seleccionada 1 de 6 células

Validação das primeiras tabelas Silver: 510 clientes, 20 000 consumos e 10 200 pedidos.

5. Controlo de qualidade na Silver

5.1 Unidades e faturas

```
df_unidades = spark.read.option("header", "true").option("inferSchema", "true").csv("Files/Bronze/unidades/unidades_consumo.csv")
df_faturas = spark.read.option("header", "true").option("inferSchema", "true").csv("Files/Bronze/faturas/faturas.csv")
```

As unidades foram deduplicadas por UnidadeID e as faturas por FaturaID, usando UltimaAtualizacao como critério. Os valores monetários foram convertidos para decimal(12,2).

```
df_faturas_silver = (
    df_faturas
    .withColumn("FaturaID", F.trim("FaturaID"))
    .withColumn("ClienteID", F.trim("ClienteID"))
    .withColumn("ValorSemIVA", F.col("ValorSemIVA").cast("decimal(12,2)"))
    .withColumn("ValorIVA", F.col("ValorIVA").cast("decimal(12,2)"))
    .withColumn("ValorTotal", F.col("ValorTotal").cast("decimal(12,2)"))
)
```

5.2 Integridade referencial

```
clientes = spark.table("silver_clientes")
unidades = spark.table("silver_unidades")
faturas = spark.table("silver_faturas")

unidades_sem_cliente = unidades.join(clientes, "ClienteID", "left_anti")
faturas_sem_cliente = faturas.join(clientes, "ClienteID", "left_anti")
```

Resultado: 0 unidades sem cliente e 1 fatura sem cliente. A fatura inválida foi FAT00000701, associada a um ClienteID inexistente.

	ABC FaturaID	ABC ClienteID	DataEmissao	12 ValorTotal	ABC EstadoPagamento
1	FAT00000701	CLI_INEXISTEN...	2025-01-05	320.03	Pago

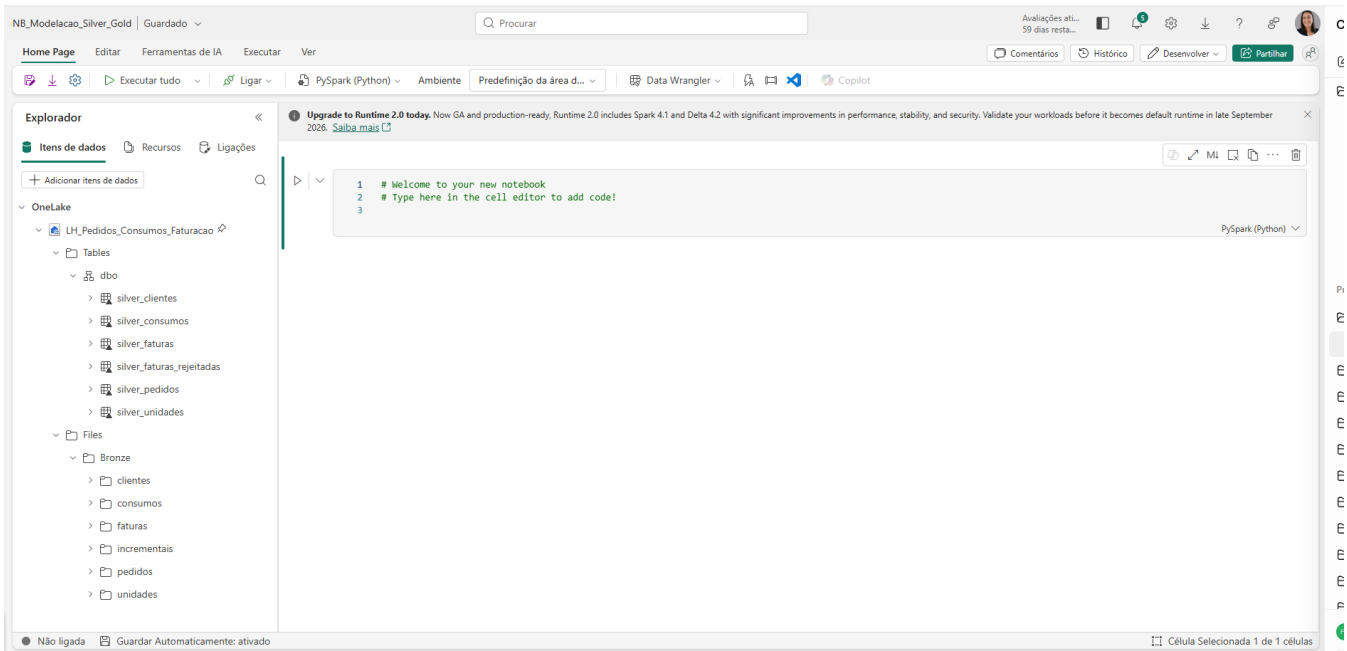
Fatura com ClienteID inexistente detetada pelo controlo de integridade referencial.

5.3 Quarentena de faturas

```
clientes_validos = spark.table("silver_clientes").select("ClienteID").distinct()
df_faturas_validas = df_faturas_silver.join(clientes_validos, "ClienteID", "left_semi")
df_faturas_rejeitadas = (
    df_faturas_silver.join(clientes_validos, "ClienteID", "left_anti")
    .withColumn("MotivoRejeicao", F.lit("ClienteID inexistente em silver_clientes"))
    .withColumn("DataRejeicao", F.current_timestamp())
)
df_faturas_validas.write.format("delta").mode("overwrite").saveAsTable("silver_faturas")
df_faturas_rejeitadas.write.format("delta").mode("overwrite").saveAsTable("silver_faturas_rejeitadas")
```

6. Modelação Silver → Gold

Foi criado um novo notebook, NB_Modelacao_Silver_Gold. O Lakehouse utiliza o esquema dbo, pelo que as tabelas foram referenciadas explicitamente como dbo.nome_tabela.



Novo notebook Gold associado ao Lakehouse e às tabelas Silver no esquema dbo.

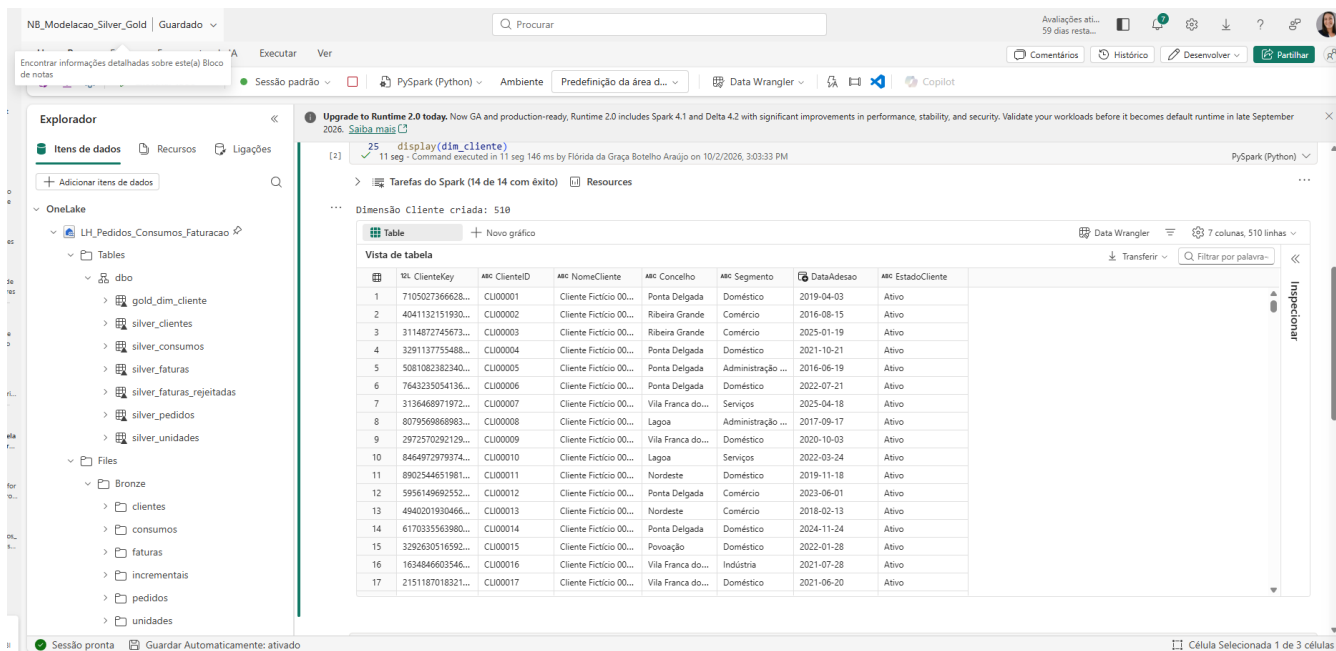
6.1 Carregar Silver

```
clientes = spark.table("dbo.silver_clientes")
unidades = spark.table("dbo.silver_unidades")
consumos = spark.table("dbo.silver_consumos")
faturas = spark.table("dbo.silver_faturas")
pedidos = spark.table("dbo.silver_pedidos")
```

6.2 Dimensão Cliente

```
dim_cliente = clientes.select(
    F.abs(F.xxhash64("ClienteID")).cast("long").alias("ClienteKey"),
    "ClienteID", "NomeCliente", "Concelho", "Segmento", "DataAdesao", "EstadoCliente"
)
dim_cliente.write.format("delta").mode("overwrite").option("overwriteSchema", "true").saveAsTable("dbo.gold_dim_cliente")
```

Dimensão Cliente criada com 510 registros e chave substituta ClienteKey.



Dimensão Cliente criada com 510 registros e chave substituta ClienteKey.

6.3 Dimensão Unidade

```
dim_unidade = (
    unidades.join(dim_cliente.select("ClienteID", "ClienteKey"), "ClienteID", "left")
    .select(
        F.abs(F.xxhash64("UnidadeID")).cast("long").alias("UnidadeKey"),
        "UnidadeID", "ClienteKey", "ClienteID", "TipoUnidade", "Concelho",
        "EstadoUnidade", "DataAtivacao"
    )
)
dim_unidade.write.format("delta").mode("overwrite").saveAsTable("dbo.gold_dim_unidade")
```

6.4 Dimensão Data

```
datas_modelo = (
    consumos.select(F.to_date("DataLeitura").alias("Data"))
    .union(faturas.select(F.to_date("DataEmissao").alias("Data")))
    .union(faturas.select(F.to_date("DataVencimento").alias("Data")))
    .union(faturas.select(F.to_date("DataPagamento").alias("Data")))
    .union(pedidos.select(F.to_date("DataAbertura").alias("Data")))
    .union(pedidos.select(F.to_date("DataConclusao").alias("Data")))
    .filter(F.col("Data").isNotNull())
)

limites = datas_modelo.agg(F.min("Data").alias("min"), F.max("Data").alias("max")).first()
dim_data = spark.sql(f"SELECT EXplode(SEQUENCE(TO_DATE('{limites['min']}'), TO_DATE('{limites['max']}'), INTERVAL 1 DAY)) AS Data")
dim_data = (dim_data
    .withColumn("DataKey", F.date_format("Data", "yyyyMMdd").cast("int"))
    .withColumn("Ano", F.year("Data"))
    .withColumn("Trimestre", F.quarter("Data"))
    .withColumn("MesNumero", F.month("Data"))
    .withColumn("AnoMes", F.date_format("Data", "yyyy-MM"))
    .withColumn("Dia", F.dayofmonth("Data")))
dim_data.write.format("delta").mode("overwrite").saveAsTable("dbo.gold_dim_data")
```

6.5 Facto Consumo

```
fact_consumo = (
    consumos.join(dim_unidade.select("UnidadeID", "UnidadeKey", "ClienteKey"), "UnidadeID", "left")
    .withColumn("DataLeituraKey", F.date_format("DataLeitura", "yyyyMMdd").cast("int"))
    .select("ConsumoID", "UnidadeKey", "ClienteKey", "DataLeituraKey", "UnidadeID",
        "DataLeitura", "Periodo", "QuantidadeKWh", "TipoLeitura")
)
```

6.6 Facto Faturação

```
fact_faturacao = (
    faturas.join(dim_cliente.select("ClienteID", "ClienteKey"), "ClienteID", "left")
    .withColumn("DataEmissaoKey", F.date_format("DataEmissao", "yyyyMMdd").cast("int"))
    .withColumn("DataVencimentoKey", F.date_format("DataVencimento", "yyyyMMdd").cast("int"))
```

```

        .withColumn("DataPagamentoKey", F.when(F.col("DataPagamento").isNotNull(), F.date_format("DataPagamento", "yyyyMMdd").cast("int"))
        .withColumn("DiasAtePagamento", F.datediff("DataPagamento", "DataEmissao"))
    )

```

6.7 Facto Pedido

```

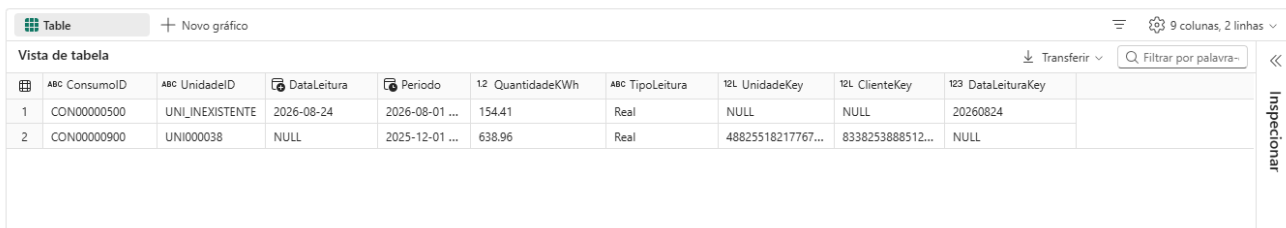
fact_pedido = (
    pedidos.join(dim_unidade.select("UnidadeID", "UnidadeKey", "ClienteKey"), "UnidadeID", "left")
    .withColumn("DataAberturaKey", F.date_format("DataAbertura", "yyyyMMdd").cast("int"))
    .withColumn("DataConclusaoKey", F.when(F.col("DataConclusao").isNotNull(), F.date_format("DataConclusao", "yyyyMMdd").cast("int"))
    .withColumn("DuracaoHoras", F.when(F.col("DataConclusao").isNotNull(),
        F.round((F.unix_timestamp("DataConclusao") - F.unix_timestamp("DataAbertura")) / 3600, 2)))
)

```

7. Qualidade e quarentena na Gold

7.1 Consumos rejeitados

Foram detetados dois registos distintos: CON00000500 com UNI_INEXISTENTE e CON00000900 com DataLeitura nula.



The screenshot shows a table with 11 columns: ABC ConsumoID, ABC UnidadeID, DataLeitura, Período, 12 QuantidadeKWh, ABC TipoLeitura, 12L UnidadeKey, 12L ClienteKey, 123 DataLeituraKey, and an empty column. There are two rows of data. The first row has values: CON00000500, UNI_INEXISTENTE, 2026-08-24, 2026-08-01 ..., 154.41, Real, NULL, NULL, 20260824. The second row has values: CON00000900, UNI000038, NULL, 2025-12-01 ..., 638.96, Real, 48825518217767..., 8338253888512..., NULL. The interface includes a 'Table' tab, a '+ Novo gráfico' button, a 'Vista de tabela' header, a 'Transferir' button, a search bar 'Filtrar por palavra-', and an 'Inspeccionar' button on the right.

	ABC ConsumoID	ABC UnidadeID	DataLeitura	Período	12 QuantidadeKWh	ABC TipoLeitura	12L UnidadeKey	12L ClienteKey	123 DataLeituraKey	
1	CON00000500	UNI_INEXISTENTE	2026-08-24	2026-08-01 ...	154.41	Real	NULL	NULL	20260824	
2	CON00000900	UNI000038	NULL	2025-12-01 ...	638.96	Real	48825518217767...	8338253888512...	NULL	

Dois consumos rejeitados: uma unidade inexistente e uma data de leitura nula.

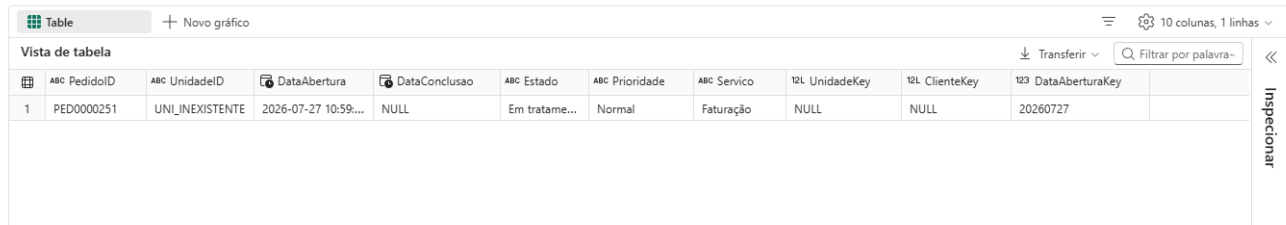
```
fact_consumo_valido = fact_consumo.filter(
    F.col("UnidadeKey").isNotNull() & F.col("ClienteKey").isNotNull() & F.col("DataLeituraKey").isNotNull())

fact_consumo_rejeitado = (fact_consumo.filter(
    F.col("UnidadeKey").isNull() | F.col("ClienteKey").isNull() | F.col("DataLeituraKey").isNull())
    .withColumn("MotivoRejeicao", F.concat_ws(" ",
        F.when(F.col("UnidadeKey").isNull(), F.lit("UnidadeID inexistente")),
        F.when(F.col("ClienteKey").isNull(), F.lit("Cliente não identificado")),
        F.when(F.col("DataLeituraKey").isNull(), F.lit("DataLeitura nula ou inválida"))))

fact_consumo_valido.write.format("delta").mode("overwrite").saveAsTable("dbo.gold_fact_consumo")
fact_consumo_rejeitado.write.format("delta").mode("overwrite").saveAsTable("dbo.gold_fact_consumo_rejeitados")
```

7.2 Pedidos rejeitados

Foi detetado o pedido PED0000251 com UNI_INEXISTENTE. O pedido tinha data de abertura válida, mas não podia ser relacionado com as dimensões Unidade e Cliente.



The screenshot shows a table with 11 columns: ABC PedidoID, ABC UnidadeID, DataAbertura, DataConclusao, ABC Estado, ABC Prioridade, ABC Servico, 12L UnidadeKey, 12L ClienteKey, 123 DataAberturaKey, and an empty column. There is one row of data: PED0000251, UNI_INEXISTENTE, 2026-07-27 10:59:..., NULL, Em tratame..., Normal, Faturação, NULL, NULL, 20260727. The interface includes a 'Table' tab, a '+ Novo gráfico' button, a 'Vista de tabela' header, a 'Transferir' button, a search bar 'Filtrar por palavra-', and an 'Inspeccionar' button on the right.

	ABC PedidoID	ABC UnidadeID	DataAbertura	DataConclusao	ABC Estado	ABC Prioridade	ABC Servico	12L UnidadeKey	12L ClienteKey	123 DataAberturaKey	
1	PED0000251	UNI_INEXISTENTE	2026-07-27 10:59:...	NULL	Em tratame...	Normal	Faturação	NULL	NULL	20260727	

Pedido com UnidadeID inexistente, enviado para quarentena.

```
fact_pedido_valido = fact_pedido.filter(
    F.col("UnidadeKey").isNotNull() & F.col("ClienteKey").isNotNull() & F.col("DataAberturaKey").isNotNull())

fact_pedido_rejeitado = (fact_pedido.filter(
    F.col("UnidadeKey").isNull() | F.col("ClienteKey").isNull() | F.col("DataAberturaKey").isNull())
    .withColumn("MotivoRejeicao", F.concat_ws(" ",
        F.when(F.col("UnidadeKey").isNull(), F.lit("UnidadeID inexistente")),
        F.when(F.col("ClienteKey").isNull(), F.lit("Cliente não identificado")),
        F.when(F.col("DataAberturaKey").isNull(), F.lit("DataAbertura nula ou inválida"))))

fact_pedido_valido.write.format("delta").mode("overwrite").saveAsTable("dbo.gold_fact_pedido")
fact_pedido_rejeitado.write.format("delta").mode("overwrite").saveAsTable("dbo.gold_fact_pedido_rejeitados")
```

8. Erros encontrados e resolução

Erro/situação	Causa	Resolução
Item com Schemas e Security	Foi associado o SQL endpoint em vez do Lakehouse	Desvincular e associar o item que mostra Tables e Files
0 duplicados em clientes	IDs visualmente semelhantes, mas distintos	Confirmar com groupBy e count; não eliminar registos válidos
1 fatura sem cliente	ClienteID fictício inexistente	Quarentena em silver_faturas_rejeitadas
1 UnidadeKey nula e 1 DataKey Null	Dados consumos inválidos diferentes	Quarentena de 2 registos na Gold
1 pedido sem UnidadeKey	UNI_INEXISTENTE	Quarentena em gold_fact_pedido_rejeitados
NameError: fact_pedido_valido	Célula de gravação executada antes da criação das tabelas	Reorganizar as células pela ordem: construir, separar, gravar

8.1 Erro NameError

```

1 (
2     fact_pedido_valido.write
3     .format("delta")
4     .mode("overwrite")
5     .option("overwriteSchema", "true")
6     .saveAsTable("dbo.gold_fact_pedido")
7 )
8
9 (
10    fact_pedido_rejeitado.write
11    .format("delta")
12    .mode("overwrite")
13    .option("overwriteSchema", "true")
14    .saveAsTable("dbo.gold_fact_pedido_rejeitados")
15 )
16
17 print(
18     "gold_fact_pedido:",
19     spark.table("dbo.gold_fact_pedido").count()
20 )
21
22 print(
23     "gold_fact_pedido_rejeitados:",
24     spark.table("dbo.gold_fact_pedido_rejeitados").count()
25 )

```

[16] 1 seg - Command executed in 1 seg 346 ms by Flórida da Graça Botelho Araújo on 10/2/2026, 3:16:30 PM

PySpark (Python)

> Diagnóstico 1

NameError
name 'fact_pedido_valido' is not defined

Diagnosticar com o Copilot | Mostrar traceback

Erro NameError causado pela execução das células fora de ordem.

O erro não danificou tabelas: a gravação não foi iniciada. A correção consistiu em executar primeiro a célula que define fact_pedido_valido e fact_pedido_rejeitado.

9. Validação final

```
tabelas_gold = [
    "dbo.gold_dim_cliente", "dbo.gold_dim_unidade", "dbo.gold_dim_data",
    "dbo.gold_fact_consumo", "dbo.gold_fact_consumo_rejeitados",
    "dbo.gold_fact_faturacao", "dbo.gold_fact_pedido",
    "dbo.gold_fact_pedido_rejeitados"
]
for tabela in tabelas_gold:
    print(f"{tabela}: {spark.table(tabela).count()}")
```

Tabela	Registos	Estado
gold_dim_cliente	510	Válida
gold_dim_unidade	800	Válida
gold_dim_data	727	Válida
gold_fact_consumo	19 998	Válida
gold_fact_consumo_rejeitados	2	Quarentena
gold_fact_faturacao	9 999	Válida
gold_fact_pedido	10 199	Válida
gold_fact_pedido_rejeitados	1	Quarentena

A validação final confirma que a camada Gold está consistente e pronta para consumo analítico. Os registos inválidos permanecem disponíveis para auditoria sem contaminar os indicadores.

10. Próximos passos

- Criar o modelo semântico SM_Pedidos_Consumos_Faturacao em modo Direct Lake.
- Incluir apenas as seis tabelas Gold analíticas; excluir Silver e tabelas de rejeitados.
- Definir relações um-para-muitos entre dimensões e factos.
- Criar medidas DAX para consumo, faturação, pagamentos, pedidos e tempos de resolução.
- Construir relatório Power BI com páginas Executiva, Consumos, Faturação, Pedidos e Qualidade.
- Orquestrar os notebooks num pipeline e configurar execução agendada.
- Adicionar logging de execução, contagem de registos e alertas de qualidade.

Fim do guia.